DBMS LAB – TASK 7 – CSE 3

SQL Views using the Taxation Database

1. Aim

To understand and apply SQL Views to simplify, secure, and reuse frequently required SQL queries.

2. Objective

By the end of this laboratory experiment, students should be able to:

- Understand the concept of SQL Views.
- Create views using SELECT queries.
- Retrieve data from views using SELECT statements.
- Create views using joins and aggregate functions.
- Understand the advantages and limitations of views.
- Solve real-world taxation queries using views for taxation analysis.

3. Concepts Covered

- SQL Views
- Creating Views
- Selecting from Views
- Views with WHERE Conditions
- Views with JOINs
- Views with Aggregate Functions
- Updating / Dropping Views
- View-based Analysis
- View Execution

4. Prerequisites

Students should already know:

- `SELECT`, `FROM`, `WHERE`

- Comparison Operators
- Logical Operators
- Aggregate Functions
- `GROUP BY`
- `HAVING`
- `ORDER BY`
- `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`
- Table aliases

These concepts were covered in the previous laboratories. Task 7 introduced subqueries and nested queries. This task builds on those concepts by presenting reusable SQL Views.

---

5. Database to be Used

Continue using the existing database:

USE taxation_db;

Existing tables:

- `Taxpayer`
- `Income_Record`
- `Income_Category`
- `Financial_Year`

No new tables are required.

Insert if any additional data required as per Queries

---

6. Introduction to SQL Views

A view is a virtual table created from the result of a SELECT query.

6. Introduction to SQL Views

A view is a virtual table created from the result of a SELECT query. A view stores the definition of a query and can be queried like a table. The underlying tables remain the source of the data.

General Syntax – CREATE VIEW

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example – Create a basic view

Create a view containing taxpayers whose occupation is Business Owner:

```
CREATE VIEW business_owners AS
SELECT taxpayer_id, taxpayer_name, occupation
FROM Taxpayer
WHERE occupation = 'Business Owner';
```

The view can then be queried like a table:

```
SELECT *
FROM business_owners;
```

Important: The SELECT query defines the view; the view itself is referenced by its view name.

7. Creating and Querying a View

A view is created using CREATE VIEW and its stored result is accessed using SELECT.

General Syntax – Querying a View

```
SELECT column1, column2, ...
FROM view_name
WHERE condition
ORDER BY column_name;
```

Example – Query a view with a condition:

```
SELECT taxpayer_id, income_amount
FROM high_income_view
WHERE income_amount > 100000;
```

8. Views with WHERE Conditions

A view can contain a WHERE condition to restrict the rows available through the view.

General Syntax

```
CREATE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Example – Business Owner taxpayers:

```
CREATE VIEW business_owner_view AS
SELECT taxpayer_id, taxpayer_name, occupation
FROM Taxpayer
WHERE occupation = 'Business Owner';
```

Querying the view:

```
SELECT *
FROM business_owner_view;
```

9. Views with JOINs

A view can combine columns from two or more tables using JOINs. This is useful when information required by users is distributed across related tables.

General Syntax – View with JOIN

```
CREATE VIEW view_name AS
SELECT t.column1, i.column2, c.column3
FROM Table1 t
INNER JOIN Table2 i ON t.key = i.key
INNER JOIN Table3 c ON i.key = c.key
WHERE condition;
```

Example – Income records with taxpayer and category details:

```
CREATE VIEW income_details_view AS
SELECT t.taxpayer_id, t.taxpayer_name,
i.income_amount, c.category_name
FROM Taxpayer t
INNER JOIN Income_Record i
ON t.taxpayer_id = i.taxpayer_id
INNER JOIN Income_Category c
ON i.category_id = c.category_id;
```

Querying the JOIN view:

```
SELECT *
FROM income_details_view
WHERE category_name = 'Business';
```

10. Views with Aggregate Functions

Aggregate functions such as AVG(), MAX(), MIN(), SUM(), and COUNT() can be used inside a view definition.

General Syntax – Aggregate View

```
CREATE VIEW view_name AS
SELECT group_column, AGGREGATE_FUNCTION(column)
FROM table_name
GROUP BY group_column;
```

Example – Total income by taxpayer:

```
CREATE VIEW taxpayer_total_income AS
SELECT taxpayer_id, SUM(income_amount) AS total_income
FROM Income_Record
GROUP BY taxpayer_id;
```

Querying the aggregate view:

```
SELECT *
FROM taxpayer_total_income
WHERE total_income > 100000;
```

Example – Average income:

```
CREATE VIEW average_income_view AS
SELECT AVG(income_amount) AS average_income
FROM Income_Record;
```

```
SELECT *
FROM average_income_view;
```

Example – Highest income using an aggregate function:

```
CREATE VIEW highest_income_view AS
SELECT *
FROM Income_Record
WHERE income_amount = (
SELECT MAX(income_amount)
FROM Income_Record
);
```

The important distinction is that **CREATE VIEW ... AS** is the view syntax, while MAX(), AVG(), SUM(), etc. are functions used inside the SELECT query that defines the view.

11. Views with Subqueries and Multi-Row Operators

A view can contain a subquery in its SELECT statement. Single-row and multi-row operators can therefore be used within a view definition.

Example – Income greater than the average income:

```
CREATE VIEW above_average_income AS
SELECT taxpayer_id, income_amount
FROM Income_Record
WHERE income_amount > (
SELECT AVG(income_amount)
FROM Income_Record
);
```

Query the view:

```
SELECT *
FROM above_average_income;
```

Example – Taxpayers with Business income using IN:

```
CREATE VIEW business_income_taxpayers AS
SELECT *
FROM Taxpayer
WHERE taxpayer_id IN (
SELECT taxpayer_id
FROM Income_Record
WHERE category_id IN (
SELECT category_id
FROM Income_Category
WHERE category_name = 'Business'
)
);
```

12. Modifying, Replacing, and Dropping Views

The definition of a view can be replaced or altered, and a view can be removed when it is no longer required.

General Syntax – CREATE OR REPLACE VIEW

```
CREATE OR REPLACE VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

General Syntax – ALTER VIEW (MySQL)

```
ALTER VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

General Syntax – DROP VIEW

```
DROP VIEW view_name;
```

Example:

```
DROP VIEW IF EXISTS business_owners;
```

13. Laboratory Activities

Part A – Basic Views

Level 1 – Understanding

Task 1: Create a view displaying the income record having the highest income.
Hint: Use MAX() in the view definition.

Task 2: Create a view displaying the income record having the lowest income.
Hint: Use MIN() in the view definition.

Task 3: Create a view displaying income records greater than the average income.
Hint: Use AVG() in the view definition.

Task 4: Create a view containing income records equal to the highest recorded income.
Hint: Use MAX() with =.

Task 5: Create a view containing taxpayers whose occupation is Business Owner.
Hint: Use a WHERE condition.

Level 2 – Application

Task 1: Create a view displaying taxpayers who have at least one income record.

Task 2: Create a view displaying taxpayers who have income in the Business category.

Task 3: Create a view displaying income records belonging to the financial year 2025-2026.
Hint: Use a JOIN with Financial_Year.

Task 4: Create a view displaying income records whose amount is greater than the minimum Business income.

Task 5: Create a view displaying income records whose amount is less than the maximum Salary income.

Task 7: Create a view displaying taxpayers who have income records greater than the average income.

Task 7: Create a view displaying income categories that have at least one income record.

Task 8: Create a view displaying taxpayers who have no income records in the Investment category.

Level 3 – Medium to Advanced

Task 1: Create a view displaying the taxpayer having the highest recorded income.
Hint: Use MAX().

Task 2: Create a view displaying all income records having an amount greater than the average Business income.

Task 3: Create a view displaying taxpayers whose total income is greater than the average total income of all taxpayers.

Task 4: Create a view displaying income records having an amount greater than at least one Investment income record.

Task 5: Create a view displaying income records having an amount greater than every Investment income record.

Task 7: Create a view displaying the income category that contains the highest income record.

Task 7: Create a view displaying the financial year having the highest total income.
Hint: Use SUM() and GROUP BY.

Task 8: Create a view displaying taxpayers whose total recorded income is greater than the average total income of taxpayers.

---

14. Real-World Taxation Analysis using Views

Task 1 Identify the taxpayer who has the highest individual income.

Hint: Use `MAX()` with a subquery.

Task 2 Identify taxpayers whose income is above the overall average income.

Hint: Use `AVG()` with a subquery.

Task 3 Identify the income category containing the highest income record.

Hint: Use `MAX()` and a nested query.

Task 4 Identify taxpayers who have Business income but no Investment income.

Hint: Use Views with WHERE Conditions.

Task 5 Identify income records whose amount is greater than every Investment income record. **Hint:** Use `ALL`.

Task 6 Identify income records whose amount is greater than at least one Investment income record. **Hint:** Use `ANY`.

Task 7 Display the taxpayer(s) having the highest total income.

Hint: Use `SUM()`, `GROUP BY`, and a subquery.

Task 8 Generate a list of income records whose amount is above the average income of their corresponding category.

Hint: This introduces the idea of comparing a record with a calculated group value; **do not use a correlated subquery yet**.

---

15. Important Concept

Students should understand the progression:

Individual Records
↓
Aggregate Functions
↓
GROUP BY
↓
HAVING
↓
ORDER BY
↓
Subquery
↓
Single-Row Subquery
↓
Multi-Row Subquery
↓
IN / NOT IN
↓
ANY / ALL
↓
Nested Analysis

The key idea is:

Outer Query → asks the final question
Inner Query → provides the value(s) required to answer it

---

16. Task 7 Submission

Students should create:

```
StudentID-repo
└── Week-7-Lab
        └── week7_views.sql
```